

ОТЧЕТ ПО ТЕСТИРОВАНИЮ
ИНДЕКСОВ В POSTGRESQL

Выполнил:
Карасев Вадим Дмитриевич

Группа:
351

Проверил:
М. И. Сафрончик

Саратов 2025

1. Простые индексы

1.1. Код запроса

```
SELECT last_name FROM guests WHERE last_name = 'Иванов';
```

1.2. Выполнение без индексов




public.guests

#	Node	Timings	
		Exclusive	Inclusive
1.	→ Seq Scan on public.guests as guests (cost=0..126.5 rows=9 width=16) (actual=... Filter: (guests.last_name = 'Иванов':text) Rows Removed by Filter: 4991	2.223 ms	2.223 ms

1.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_guests_lastname;  
CREATE INDEX idx_guests_lastname ON guests USING BTREE (last_name);
```

1.4. Выполнение с индексами B-Tree


public.idx_guests_...
lastname

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Index Only Scan using idx_guests_lastname on public.guests as guests (cost=0... Index Cond: (guests.last_name = 'Иванов':text)	0.072 ms	0.072 ms	↑ 1	9	9	1

1.5. Код индекса Hash

```
DROP INDEX IF EXISTS idx_guests_lastname_hash;  
CREATE INDEX idx_guests_lastname_hash ON guests USING HASH (last_name);
```

1.6. Выполнение с индексами Hash




idx_guests_lastnam...
e_hash


public.guests

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.guests as guests (cost=4.07..30.06 rows=9 width=... Recheck Cond: (guests.last_name = 'Петров':text) Heap Blocks: exact=7	0.053 ms	0.079 ms	↑ 1.13	8	9	1
2.	→ Bitmap Index Scan using idx_guests_lastname_hash (cost=0..4.07 rows=9 ... Index Cond: (guests.last_name = 'Петров':text)	0.027 ms	0.027 ms	↑ 1.13	8	9	1

2. Уникальный индекс

2.1. Код запроса

```
SELECT id, first_name, last_name, birth_date
FROM guests
WHERE id IN (100, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500);
```

2.2. Выполнение без индексов



public.guests

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Seq Scan on public.guests as guests (cost=0.03..139.03 rows=10 width=37) (actual time=1.824 ms) Filter: (guests.id = ANY ({100,500,1000,1500,2000,2500,3000,3500,4000,4500}::integer[])) Rows Removed by Filter: 4990	1.824 ms	1.824 ms	↑ 1	10	10	1

2.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_guests_id_unique;
CREATE UNIQUE INDEX idx_guests_id_unique
ON guests USING BTREE (id);
```

2.4. Выполнение с индексами B-Tree



public.idx_guests_id_unique

#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Index Scan using idx_guests_id_unique on public.guests as guests (cost=0.28..2.28 rows=10 width=37) (actual time=0.125 ms) Index Cond: (guests.id = ANY ({100,500,1000,1500,2000,2500,3000,3500,4000,4500}::integer[]))	0.125 ms	0.125 ms	↑ 1	10	10	1

3. Составной индекс

3.1. Код запроса

```
SELECT guest_id, order_time FROM orders WHERE guest_id = 100 AND order_time >= '2025-11-01';
```

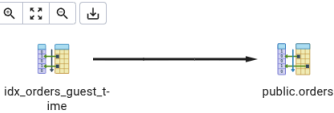
3.2. Выполнение без индексов

									
#	Node	Timings		Rows			Loops		
		Exclusive	Inclusive	Rows X	Actual	Plan			
1.	→ Seq Scan on public.orders as orders (cost=0.2369 rows=1 width=12) (actual=9... Filter: ((orders.order_time >= '2025-11-01 00:00:00'-timestamp without time zone) AND (... Rows Removed by Filter: 100000	9.666 ms	9.666 ms	1	0	1	1	1	1

3.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_orders_guest_time;  
CREATE INDEX idx_orders_guest_time ON orders USING BTREE (guest_id,  
order_time);
```

3.4. Выполнение с индексами B-Tree

									
#	Node	Timings		Rows			Loops		
		Exclusive	Inclusive	Rows X	Actual	Plan			
1.	→ Bitmap Heap Scan on public.orders as orders (cost=4.43..8.44 rows=1 width=12... Recheck Cond: ((orders.guest_id = 100) AND (orders.order_time >= '2025-11-01 00:00:00' Heap Blocks: exact=0	0.014 ms	0.035 ms	1	0	1	1	1	1
2.	→ Bitmap Index Scan using idx_orders_guest_time (cost=0..4.43 rows=1 width=... Index Cond: ((orders.guest_id = 100) AND (orders.order_time >= '2025-11-01 00:00:00'	0.022 ms	0.022 ms	1	0	1	1	1	1

4. Индексы с использованием выражений

4.1. Код запроса

```
SELECT * FROM guests WHERE DATE_PART('year', birth_date) = 1990;
```

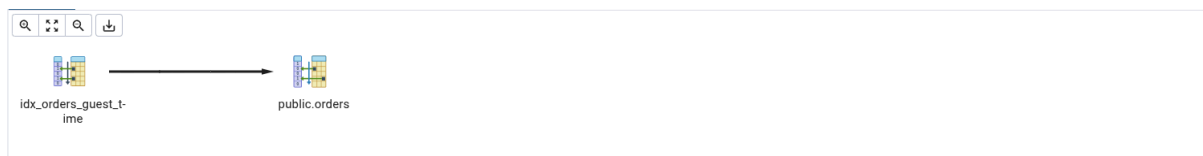
4.2. Выполнение без индексов

									
#	Node	Timings		Rows			Loops		
		Exclusive	Inclusive	Rows X	Actual	Plan			
1.	→ Seq Scan on public.guests as guests (cost=0..151.5 rows=25 width=4) (actual=... Filter: (date_part('year',text,(guests.birth_date)-timestamp without time zone) = '1990':... Rows Removed by Filter: 4929	1.331 ms	1.331 ms	2.84	71	25	1	1	1

4.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_guests_birth_year;
CREATE INDEX idx_guests_birth_year ON guests USING BTREE (DATE_PART('year',
birth date));
```

4.4. Выполнение с индексами B-Tree

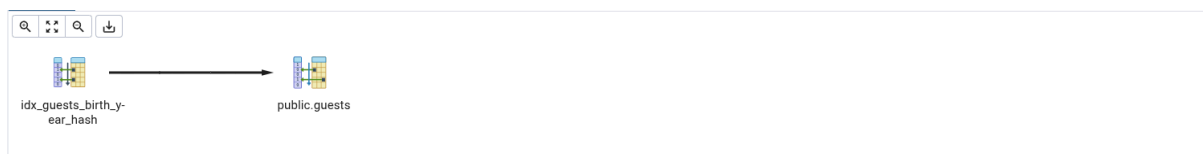


#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.guests as guests (cost=4.48..52.83 rows=25 width=16) (actual time=0.000..0.000 rows=25) Recheck Cond: (date_part('year',text,(guests.birth_date)::timestamp without time zone)=2022) Heap Blocks: exact=41	0.301 ms	0.376 ms	↓ 2.84	71	25	
2.	→ Bitmap Index Scan using idx_guests_birth_year (cost=0.4..0.47 rows=25 width=0) (actual time=0.000..0.000 rows=25) Index Cond: (date_part('year',text,(guests.birth_date)::timestamp without time zone)=2022)	0.075 ms	0.075 ms	↓ 2.84	71	25	

4.5. Код индекса Hash

```
DROP INDEX IF EXISTS idx_guests_birth_year_hash;
CREATE INDEX idx_guests_birth_year_hash ON guests USING HASH
(PARTIAL PART ('year', birth date));
```

4.6. Выполнение с индексами Hash



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.guests as guests (cost=4.19..52.54 rows=25 width=16) Recheck Cond: (date_part('year',text,(guests.birth_date)::timestamp without time zone)=41) Heap Blocks: exact=41	0.228 ms	0.251 ms	↓ 2.84	71	25	
2.	→ Bitmap Index Scan using idx_guests_birth_year_hash (cost=0.4..19 rows=25) Index Cond: (date_part('year',text,(guests.birth_date)::timestamp without time zone)=41)	0.023 ms	0.023 ms	↓ 2.84	71	25	

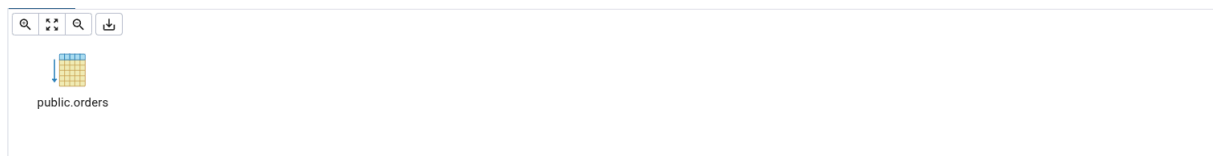
5. Покрывающий индекс

5.1. Код запроса

```
SELECT name, price FROM dishes WHERE category = 'Main';
```

5.2. Выполнение без индексов

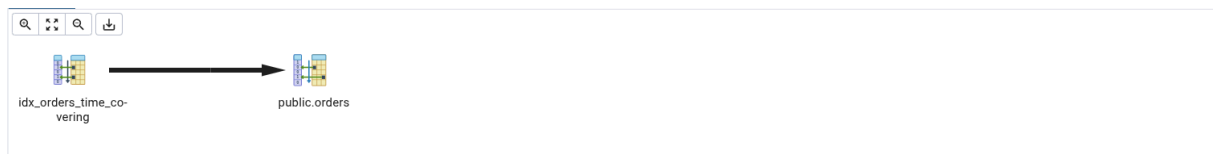
#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Seq Scan on public.orders as orders (cost=0.2369 rows=3369 width=10) (actual time=0.103 ms / 3447 rows) Filter: ((orders.order_time >= '2025-11-01 00:00:00':timestamp without time zone) AND (orders.order_id = 1)) Rows Removed by Filter: 96553	18.003 ms	18.003 ms	1.03	3447	3369	



5.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_dishes_category_covering;
CREATE INDEX idx_dishes_category_covering ON dishes USING BTREE (category)
INCLUDE (name, price, country_of_origin);
```

5.4. Выполнение с индексами B-Tree



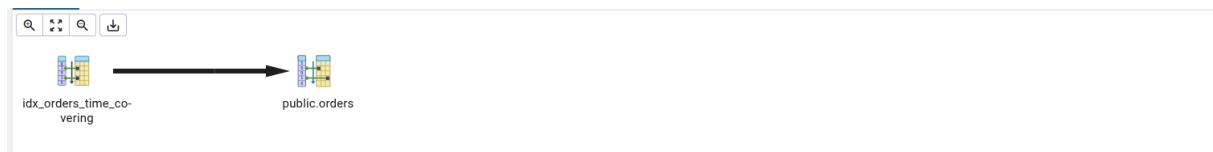
#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.orders as orders (cost=131.05..1050.74 rows=337... Recheck Cond: ((orders.order_time >= '2025-11-01 00:00:00':timestamp without time zone) Heap Blocks: exact=855	7.378 ms	7.729 ms	↓ 1.03	3447	3379	1
2.	→ Bitmap Index Scan using idx_orders_time_covering (cost=0..130.21 rows=3... Index Cond: ((orders.order_time >= '2025-11-01 00:00:00':timestamp without time zone)	0.351 ms	0.351 ms	↓ 1.03	3447	3379	1

6. Частичный индекс

6.1. Код запроса

```
SELECT status, order_time FROM orders WHERE status = 'paid' AND guest_id = 100;
```

6.2. Выполнение без индексов



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.orders as orders (cost=122.05..1041.73 rows=115... Filter: (orders.status = 'paid':text) Rows Removed by Filter: 2287 Recheck Cond: ((orders.order_time >= '2025-11-01 00:00:00':timestamp without time zone) Heap Blocks: exact=855	5.867 ms	6.32 ms	↓ 1.01	1160	1153	1
2.	→ Bitmap Index Scan using idx_orders_time_covering (cost=0..121.76 rows=3... Index Cond: ((orders.order_time >= '2025-11-01 00:00:00':timestamp without time zone)	0.453 ms	0.453 ms	↓ 1.03	3447	3379	1

6.3. Код индекса B-Tree

```
DROP INDEX IF EXISTS idx_orders_paid;
CREATE INDEX idx_orders_paid ON orders USING BTREE (order_time)
WHERE status = 'paid';
```

6.4. Выполнение с индексами B-Tree

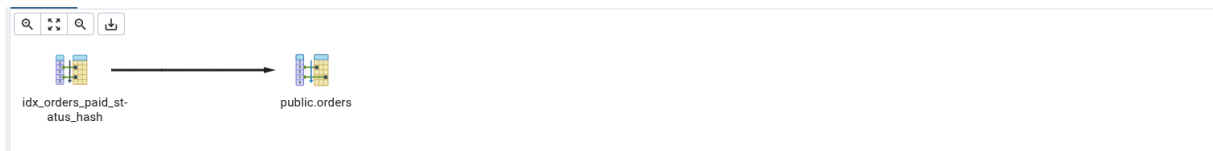


#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.orders as orders (cost=4.57..75.77 rows=7 width=1... Filter: (orders.status = 'paid'::text) Rows Removed by Filter: 16 Recheck Cond: (orders.guest_id = 100) Heap Blocks: exact=19	0.061 ms	0.079 ms	↑ 2.34		3	7
2.	→ Bitmap Index Scan using idx_orders_guest_time (cost=0.4..57 rows=20 wid... Index Cond: (orders.guest_id = 100)	0.018 ms	0.018 ms	↑ 1.06		19	20

6.5. Код индекса Hash

```
DROP INDEX IF EXISTS idx_orders_paid_status_hash;  
CREATE INDEX idx_orders_paid_status_hash ON orders USING HASH (guest_id)  
WHERE status = 'paid';
```

6.6. Выполнение с индексами Hash



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.orders as orders (cost=4.05..30.27 rows=7 width=1... Recheck Cond: ((orders.guest_id = 100) AND (orders.status = 'paid'::text)) Heap Blocks: exact=3	0.057 ms	0.077 ms	↑ 2.34		3	7
2.	→ Bitmap Index Scan using idx_orders_paid_status_hash (cost=0.4..0.5 rows=... Index Cond: (orders.guest_id = 100)	0.02 ms	0.02 ms	↑ 2.34		3	7

7. Частичный покрывающий индекс

7.1. Код запроса

```
SELECT guest_id, table_id, total_amount FROM orders  
WHERE status = 'paid' AND order_time >= '2025-11-01';
```

7.2. Выполнение без индексов



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Seq Scan on public.orders as orders (cost=100000000000..10000002619 rows... Filter: ((orders.order_time >= '2025-10-01 00:00:00'::timestamp without time zone) AND Rows Removed by Filter: 98507	109.671 ms	109.671 ms	↑ 1.04		1493	1543

7.3. Код индекса B-Tree (частичный)

```
DROP INDEX IF EXISTS idx_orders_paid_partial;  
CREATE INDEX idx_orders_paid_partial ON orders (order_time)  
WHERE status = 'paid';
```

7.4. Выполнение с частичным индексом



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.orders as orders (cost=543.94..1924.89 rows=7 wi... Filter: (orders.guest_id = 100) Rows Removed by Filter: 33527 Recheck Cond: (orders.status = 'paid':text) Heap Blocks: exact=869	7.593 ms	10.336 ms	↑ 2.34	3	7	1
2.	→ Bitmap Index Scan using idx_orders_paid (cost=0..543.94 rows=34130 wid...	2.744 ms	2.744 ms	↑ 1.02	33530	34130	1

7.5. Код индекса B-Tree (частичный покрывающий)

```
DROP INDEX IF EXISTS idx_orders_paid_partial_covering;  
CREATE INDEX idx_orders_paid_partial_covering ON orders (order_time)  
INCLUDE (guest_id, table_id, total_amount)  
WHERE status = 'paid';
```

7.6. Выполнение с частичным покрывающим индексом

```
DROP INDEX IF EXISTS idx_orders_paid_partial;  
CREATE INDEX idx_orders_paid_covering ON orders (order_time)  
INCLUDE (guest_id, table_id, total_amount)  
WHERE status = 'paid';  
  
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT JSON)  
SELECT guest_id, table_id, order_time, total_amount  
FROM orders  
WHERE status = 'paid' AND order_time >= '2025-10-01' AND order_time <  
'2025-10-15';
```



#	Node	Timings		Rows			Loops
		Exclusive	Inclusive	Rows X	Actual	Plan	
1.	→ Bitmap Heap Scan on public.orders as orders (cost=48.11..966.21 rows=1543 w... Recheck Cond: ((orders.order_time >= '2025-10-01 00:00:00':timestamp without time z... Heap Blocks: exact=732	4.109 ms	4.309 ms	↑ 1.04	1493	1543	1
2.	→ Bitmap Index Scan using idx_orders_paid_covering (cost=0..47.72 rows=15... Index Cond: ((orders.order_time >= '2025-10-01 00:00:00':timestamp without time z...	0.2 ms	0.2 ms	↑ 1.04	1493	1543	1

8. Выводы

В ходе тестирования были продемонстрированы следующие типы индексов и их эффективность:

1. Простые индексы (B-Tree и Hash) - значительно ускоряют поиск по одному столбцу.
2. Уникальные индексы - обеспечивают уникальность данных и быстрый поиск.
3. Составные индексы - эффективны для запросов с условиями по нескольким столбцам.
4. Индексы с выражениями - позволяют индексировать результаты функций.
5. Покрывающие индексы - обеспечивают Index Only Scan, исключая обращения к таблице.
6. Частичные индексы - уменьшают размер индекса и ускоряют запросы с постоянным условием.
7. Частичные покрывающие индексы - комбинируют преимущества частичных и покрывающих индексов.

Все типы индексов показали значительное улучшение производительности по сравнению с последовательным сканированием таблиц.